PUC Minas

Engenharia de Software

# Sistema de Aluguel de Carros

Modelagem do Sistema: Casos de Uso, Histórias do Usuário, Diagrama de Classes e Diagrama de Pacotes (Visão Lógica).

| | |
|---|---|
| **Disciplina** | DIAW – Desenvolvimento e Integração de Aplicações Web |
| **Atividade** | Laboratório 02 (15 pontos) – Processo de Desenvolvimento: Modelagem do sistema |
| **Professor** | João Paulo Carneiro Aramuni |
| **Integrantes** | Bernardo Guedes da Silveira |
| **Repositório GitHub** | https://github.com/Bernardo-Guedes/BernarDiesel |

## 1. Introdução

Este documento apresenta a modelagem do Sistema de Aluguel de Carros, um sistema web para apoio à gestão de aluguéis de automóveis que permite efetuar, cancelar e modificar pedidos através da Internet. A modelagem cobre os artefatos exigidos no Laboratório 02: Diagrama de Casos de Uso, Histórias do Usuário, Diagrama de Classes e Diagrama de Pacotes (Visão Lógica).

### 1.1 Visão geral do sistema

O sistema só pode ser utilizado após cadastro prévio e autenticação (login com e-mail e senha). Clientes (usuários individuais) podem introduzir, modificar, consultar e cancelar pedidos de aluguel, além de consultar seus rendimentos e dados cadastrais. Agentes (empresas e bancos), previamente cadastrados pelo administrador, podem modificar e avaliar pedidos, analisar financeiramente os clientes e executar contratos. Depois de introduzido, o pedido é analisado financeiramente pelos agentes e, em caso de parecer positivo, fica disponível para a execução do contrato. O aluguel pode estar associado a um contrato de crédito concedido por um dos bancos agentes.

### 1.2 Stack tecnológica

| Camada | Tecnologia |
|---|---|
| **Linguagem** | Java – JDK 25 |
| **Back-end** | Spring Boot 4.0.x (arquitetura MVC) |
| **Build e dependências** | Apache Maven |
| **Front-end** | React + Vite |
| **Banco de dados** | Supabase (PostgreSQL) |

| Camada | Tecnologia |
|---|---|
| **Autenticação** | Login obrigatório (e-mail e senha) |

## 1.3 Atores

| Ator | Descrição |
|---|---|
| **Cliente** | Usuário individual que contrata o aluguel. Gerencia seus dados, rendimentos (máximo 3) e pedidos. |
| **Agente** | Ator abstrato que representa as entidades que analisam pedidos e executam contratos. Especializa-se em Agente Empresa e Agente Banco. |
| **Agente Empresa** | Empresa agente. Pode ser proprietária de automóveis alugados, dependendo do tipo de contrato. |
| **Agente Banco** | Banco agente. Pode ser proprietário de automóveis e concede contratos de crédito associados ao aluguel. |
| **Administrador** | Responsável pelo cadastro prévio de agentes (empresas e bancos), garantindo que somente agentes autorizados tenham acesso ao sistema. |

## 1.4 Requisitos

### Requisitos funcionais

| ID | Requisito |
|---|---|
| **RF01** | O sistema deve permitir o cadastro prévio de clientes e o cadastro de agentes pelo administrador, além da autenticação por e-mail e senha. |
| **RF02** | O sistema deve permitir o CRUD dos dados do cliente (RG, CPF, nome, endereço e profissão), incluindo a desativação lógica da conta. |
| **RF03** | O sistema deve armazenar as entidades empregadoras e os respectivos rendimentos do cliente (máximo 3). |
| **RF04** | O cliente deve poder introduzir, modificar, consultar e cancelar pedidos de aluguel. |
| **RF05** | O cliente deve poder visualizar o status de seus pedidos. |
| **RF06** | Os agentes devem poder modificar e avaliar pedidos (análise financeira). |
| **RF07** | Pedidos com parecer positivo devem ser colocados à consideração dos agentes para execução do contrato. |
| **RF08** | O sistema deve registrar os automóveis (matrícula, ano, marca, modelo e placa) e sua propriedade (cliente, empresa ou banco), conforme o tipo de contrato. |
| **RF09** | O aluguel poderá estar associado a um contrato de crédito concedido por um banco agente. |

### Requisitos não funcionais

| ID | Requisito |
|---|---|
| **RNF01** | Sistema web, com servidor central acessível por clientes e agentes via Internet. |
| **RNF02** | Back-end em Java (JDK 25) com Spring Boot 4.0.x, usando arquitetura MVC e Maven. |
| **RNF03** | Front-end em React + Vite, consumindo a API do back-end. |

| ID | Requisito |
|---|---|
| **RNF04** | Persistência no Supabase (PostgreSQL). |
| **RNF05** | Acesso restrito a usuários cadastrados e autenticados; senhas armazenadas com hash. |
| **RNF06** | O sistema é dividido em dois subsistemas: gestão de pedidos e contratos; construção dinâmica das páginas web. |

# 2. Diagrama de Casos de Uso

O diagrama abaixo apresenta as funcionalidades do sistema do ponto de vista dos atores. Os casos de uso UC03 a UC08 pertencem ao Cliente; UC09 a UC12 pertencem aos Agentes. O ator Agente é generalizado em Agente Empresa e Agente Banco, e somente o Agente Banco interage com o UC12 (contrato de crédito).

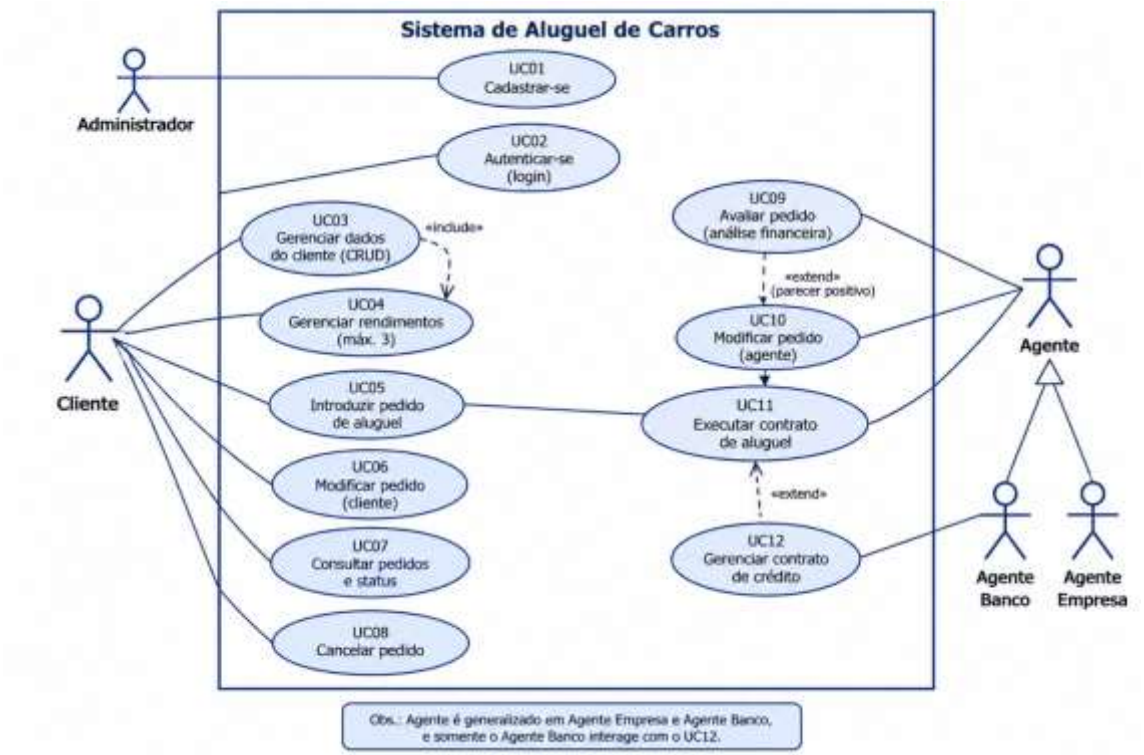



*Figura 1 – Diagrama de Casos de Uso do Sistema de Aluguel de Carros*

## 2.1 Descrição dos casos de uso

*Tabela 1 – Descrição resumida dos casos de uso*

| ID | Caso de uso | Ator(es) | Descrição |
|---|---|---|---|
| **UC01** | Cadastrar-se | Cliente, Agente | Permite o cadastro de clientes pelo próprio usuário e o cadastro prévio de agentes (empresas e bancos) pelo administrador, utilizando a ADMIN_KEY. |
| **UC02** | Autenticar-se (login) | Cliente, Agente | Valida e-mail e senha e libera o acesso conforme o perfil. |
| **UC03** | Gerenciar dados do cliente (CRUD) | Cliente | Cria, consulta e atualiza os dados de identificação (RG, CPF, nome, endereço) e a profissão. Desativa a conta. |
| **UC04** | Gerenciar rendimentos | Cliente | Inclui, altera e remove entidades empregadoras e rendimentos (máximo 3). Incluído por UC03. |
| **UC05** | Introduzir pedido de aluguel | Cliente | Cria um novo pedido informando automóvel e período. O pedido entra com status “Em análise”. |
| **UC06** | Modificar pedido (cliente) | Cliente | Altera dados de um pedido próprio enquanto ele estiver em análise. |

| ID | Caso de uso | Ator(es) | Descrição |
|---|---|---|---|
| **UC07** | Consultar pedidos e status | Cliente | Lista e detalha os pedidos do cliente e exibe o status de cada um. |
| **UC08** | Cancelar pedido | Cliente | Cancela um pedido próprio que esteja com status "Em análise" ou "Aprovado", desde que ainda não tenha sido contratado. |
| **UC09** | Avaliar pedido (análise financeira) | Agente | Analisa o pedido com base nos rendimentos do cliente e registra parecer positivo ou negativo. |
| **UC10** | Modificar pedido (agente) | Agente | Altera dados de um pedido durante a análise (por exemplo, valores e condições). |
| **UC11** | Executar contrato de aluguel | Agente | Formaliza o contrato de pedidos com parecer positivo. Estende UC09 e inclui UC12. |
| **UC12** | Gerenciar contrato de crédito | Agente Banco | Registra o crédito concedido por um banco e o associa ao contrato de aluguel. Estende UC11. |

## 2.2 Fluxo principal dos casos de uso centrais

### UC05 – Introduzir pedido de aluguel

- **Pré-condição:** cliente autenticado e com dados cadastrados.
- O cliente acessa “Novo pedido” e informa o automóvel desejado e o período de aluguel.
- O sistema valida os dados e vincula o pedido ao cliente autenticado.
- O sistema grava o pedido com status “Em análise” e exibe a confirmação.
- **Exceção:** dados inválidos ou incompletos geram mensagem de erro e o pedido não é gravado.

### UC09 – Avaliar pedido (análise financeira)

- **Pré-condição:** agente autenticado e pedido com status “Em análise”.
- O agente seleciona o pedido e consulta os dados do cliente e seus rendimentos.
- O agente registra o parecer financeiro (positivo ou negativo) com justificativa.
- Parecer positivo: o pedido passa para “Aprovado” e fica apto a ter o contrato executado (UC11).
- Parecer negativo: o pedido passa para “Reprovado”.

### UC11 – Executar contrato de aluguel

- **Pré-condição:** pedido com status “Aprovado”.
- O agente confirma a execução e define o tipo de propriedade do automóvel (cliente, empresa ou banco).
- O sistema registra o automóvel e, se houver crédito, o banco o associa (UC12).
- O sistema gera o contrato e altera o status do pedido para “Contratado”.

## 2.3 Regras de negócio

| ID | Regra de negócio |
|---|---|
| **RN01** | Todo acesso, exceto cadastro e login, exige usuário autenticado. |
| **RN02** | Cada cliente pode ter, no máximo, 3 rendimentos cadastrados. |
| **RN03** | O cliente só pode consultar, modificar e cancelar os seus próprios pedidos. |
| **RN04** | O cliente só pode modificar um pedido enquanto ele estiver com status “Em análise”. |
| **RN05** | Somente agentes avaliam pedidos; o contrato só pode ser executado após parecer positivo. |
| **RN06** | O automóvel alugado pode ser propriedade de cliente, empresa ou banco, dependendo do tipo de contrato. |
| **RN07** | O contrato de crédito é opcional e só pode ser concedido por um banco agente. |
| **RN08** | Pedidos com status "Em análise" ou "Aprovado" podem ser cancelados pelo cliente. Pedidos "Reprovados" ou "Contratados" não podem ser cancelados. |
| **RN09** | CPF do cliente, login do usuário e placa do automóvel devem ser únicos. |
| **RN10** | As datas de início e término do aluguel devem ser futuras e válidas. |
| **RN11** | Não é permitido criar pedidos com períodos sobrepostos para o mesmo automóvel. |
| **RN12** | O cancelamento de pedidos só é permitido quando o status for "Em análise" ou "Aprovado". Pedidos reprovados ou contratados não podem ser cancelados. |

# 3. Histórias do Usuário

As histórias seguem o formato “Como [ator], quero [ação], para [benefício]”. A prioridade usa a escala Alta, Média e Baixa, considerando o escopo do protótipo (autenticação, CRUD de cliente e criação/visualização de pedidos).

*Tabela 2 – Histórias do usuário com critérios de aceitação*

| ID | História | Critérios de aceitação |
|---|---|---|
| **US01** | Como cliente, quero me cadastrar no sistema, para poder utilizar os serviços de aluguel. | • Informar nome, CPF, RG, endereço, profissão, e-mail e senha.<br>• CPF e e-mail devem ser únicos.<br>• Senha armazenada com hash.<br>• Exibir confirmação ao concluir o cadastro. |
| **US02** | Como administrador, quero cadastrar agentes (empresas e bancos), para permitir que eles acessem o sistema e realizem suas operações. | • Informar tipo (empresa ou banco), nome, CNPJ, e-mail e senha.<br>• O cadastro deve exigir a ADMIN_KEY<br>• Perfil do agente definido no cadastro. |
| **US03** | Como usuário cadastrado, quero fazer login com e-mail e senha, para acessar as funcionalidades do meu perfil. | • Credenciais válidas liberam o acesso conforme o perfil.<br>• Credenciais inválidas exibem mensagem de erro.<br>• Páginas protegidas redirecionam para o login se não houver sessão.<br>• Opção de encerrar a sessão (logout). |
| **US04** | Como cliente, quero consultar, atualizar e desativar minha conta, para manter meus dados cadastrais corretos e controlar meu acesso ao sistema. | • Exibir os dados atuais do cliente logado.<br>• Validar campos obrigatórios na atualização.<br>• Desativação exige confirmação.<br>• A conta deve ser desativada sem excluir fisicamente os registros do banco de dados |
| **US05** | Como cliente, quero informar minhas entidades empregadoras e rendimentos, para fundamentar a análise financeira dos meus pedidos. | • Permitir cadastrar até 3 rendimentos.<br>• Cada rendimento tem entidade empregadora e valor.<br>• Bloquear o 4º rendimento com mensagem clara. |
| **US06** | Como cliente, quero introduzir um pedido de aluguel, para solicitar um automóvel por um período. | • Informar automóvel e período (início e fim).<br>• Pedido criado com status “Em análise”.<br>• Pedido vinculado ao cliente autenticado. |
| **US07** | Como cliente, quero modificar um pedido existente, para corrigir ou ajustar o que solicitei. | • Permitido apenas com status “Em análise”.<br>• Alterações são gravadas e exibidas na consulta. |
| **US08** | Como cliente, quero consultar meus pedidos e ver o status de cada um, para acompanhar o andamento. | • Listar apenas os pedidos do cliente logado.<br>• Exibir status: Em análise, Aprovado, Reprovado, Contratado ou Cancelado.<br>• Permitir ver os detalhes do pedido. |
| **US09** | Como cliente, quero cancelar um pedido, para desistir do aluguel quando necessário. | • Solicitar confirmação antes de cancelar.<br>• Permitir cancelamento somente nos status "Em análise" e "Aprovado".<br>• Status passa para “Cancelado”. |

| ID | História | Critérios de aceitação |
|---|---|---|
| **US10** | Como agente, quero avaliar financeiramente um pedido, para decidir se ele segue para a execução do contrato. | • Listar pedidos “Em análise”.<br>• Exibir dados e rendimentos do cliente.<br>• Registrar parecer positivo ou negativo com justificativa.<br>• Positivo → “Aprovado”; negativo → “Reprovado”. |
| **US11** | Como agente, quero modificar um pedido, para ajustar condições durante a análise. | • Permitido enquanto o pedido não estiver “Contratado” ou “Cancelado”.<br>• Alteração registrada no pedido. |
| **US12** | Como agente, quero executar o contrato de um pedido aprovado, para formalizar o aluguel. | • Apenas pedidos “Aprovado” podem ser contratados.<br>• Utilizar o automóvel previamente cadastrado, mantendo seu tipo de propriedade.<br>• Status passa para “Contratado”. |
| **US13** | Como agente banco, quero registrar um contrato de crédito associado ao aluguel, para financiar a locação do cliente. | • Disponível apenas para o perfil Banco.<br>• Informar valor financiado, taxa de juros e prazo.<br>• Crédito associado a um único contrato de aluguel. |

# 4. Diagrama de Classes

O diagrama de classes representa o modelo de domínio e será a base das entidades do back-end (pacote model). Usuário é uma classe abstrata especializada em Cliente e Agente; Agente, também abstrata, é especializada em Empresa e Banco. A interface Proprietário é realizada por Cliente, Empresa e Banco, permitindo que um automóvel pertença a qualquer um deles conforme o tipo de contrato.

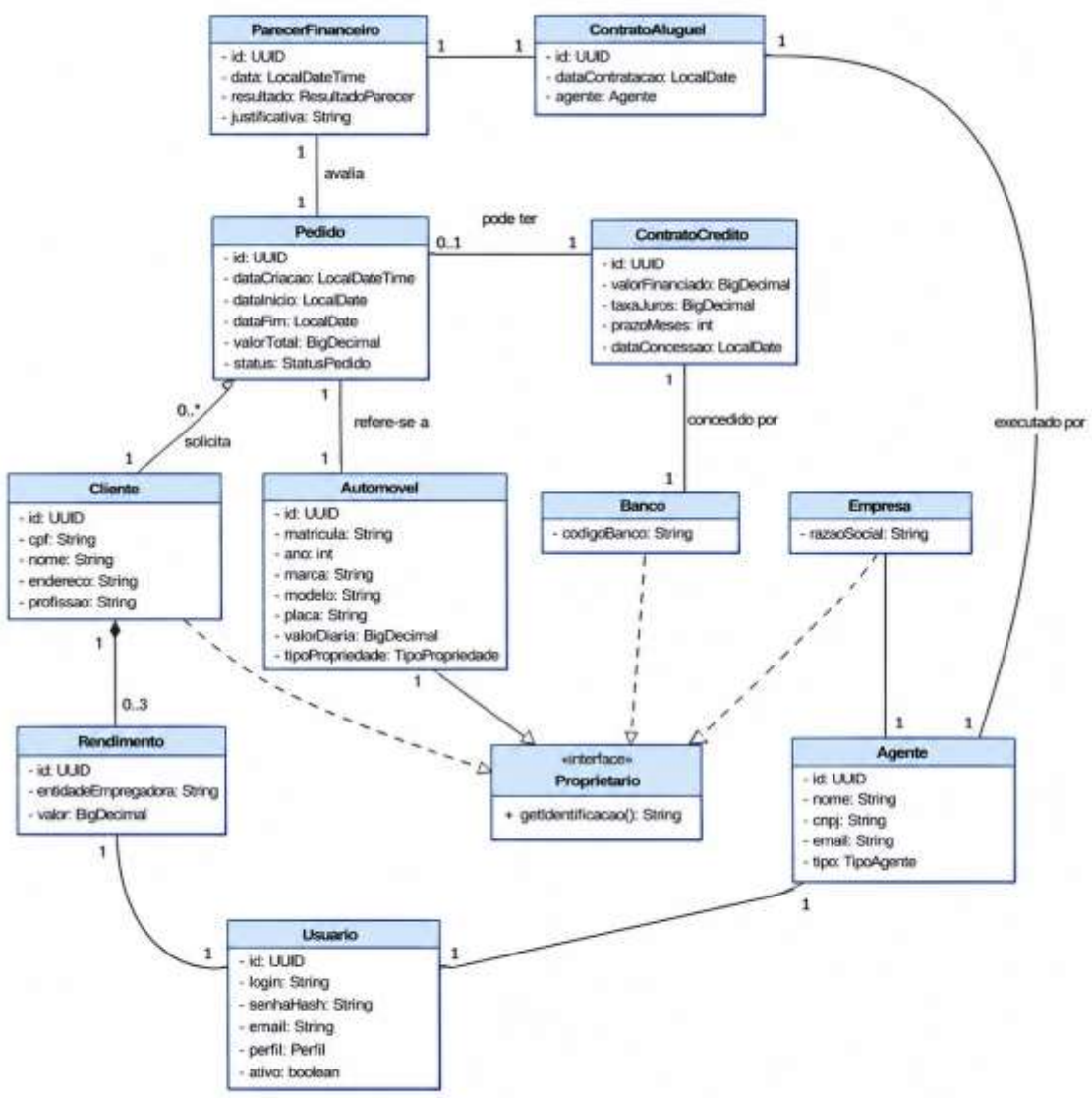



*Figura 2 – Diagrama de Classes do Sistema de Aluguel de Carros*

## 4.1 Descrição das classes

*Tabela 3 – Responsabilidades das classes*

| Classe | Responsabilidade |
|---|---|
| **Usuario (abstrata)** | Dados de acesso comuns (login, senha com hash, e-mail, perfil) e autenticação. |
| **Cliente** | Contratante individual. Armazena RG, CPF, nome, endereço e profissão e possui até 3 rendimentos. |

| Classe | Responsabilidade |
|---|---|
| **Rendimento** | Entidade empregadora e valor do rendimento auferido pelo cliente. |
| **Agente (abstrata)** | Entidade (empresa ou banco) que avalia e modifica pedidos e executa contratos. |
| **Empresa** | Agente do tipo empresa. Pode ser proprietária de automóveis. |
| **Banco** | Agente do tipo banco. Pode ser proprietário de automóveis e concede contratos de crédito. |
| **Proprietario (interface)** | Abstrai quem pode ser dono de um automóvel: Cliente, Empresa ou Banco. |
| **Pedido** | Solicitação de aluguel feita por um cliente para um automóvel, com período, valor e status. |
| **ParecerFinanceiro** | Resultado da análise financeira de um pedido, emitido por um agente. |
| **Automovel** | Veículo alugado: matrícula, ano, marca, modelo e placa; associado a um proprietário. |
| **ContratoAluguel** | Contrato originado de um pedido aprovado e executado por um agente. |
| **ContratoCredito** | Crédito concedido por um banco e associado (opcionalmente) a um contrato de aluguel. |

## 4.2 Relacionamentos e multiplicidades

*Tabela 4 – Relacionamentos do diagrama de classes*

| Relacionamento | Tipo | Multiplicidade |
|---|---|---|
| **Cliente – Rendimento** | Composição | 1 cliente possui 0..3 rendimentos |
| **Pedido – Cliente** | Associação | Cada pedido pertence a 1 cliente; um cliente tem vários pedidos |
| **Pedido – Automovel** | Associação | Cada pedido refere-se a 1 automóvel; um automóvel pode estar em vários pedidos ao longo do tempo |
| **Automovel – Proprietario** | Associação | Cada automóvel pertence a 1 proprietário (Cliente, Empresa ou Banco) |
| **ParecerFinanceiro – Pedido** | Associação | Um pedido pode ter 0..* pareceres; cada parecer avalia 1 pedido |
| **ParecerFinanceiro – Agente** | Associação | Cada parecer é emitido por 1 agente |
| **ContratoAluguel – Pedido** | Associação | Um pedido origina 0..1 contrato; cada contrato vem de 1 pedido |
| **ContratoAluguel – Agente** | Associação | Cada contrato é executado por 1 agente |
| **ContratoAluguel – ContratoCredito** | Associação | Um contrato de aluguel pode ter 0..1 contrato de crédito |
| **ContratoCredito – Banco** | Associação | Cada crédito é concedido por 1 banco |
| **Cliente, Agente → Usuario; Empresa, Banco → Agente** | Generalização | Herança de dados de acesso e de agente |

| Relacionamento | Tipo | Multiplicidade |
|---|---|---|
| **Cliente, Empresa, Banco → Proprietario** | Realização | Implementação da interface Proprietario |

## 4.3 Enumerações

*Tabela 5 – Enumerações utilizadas no modelo*

| Enumeração | Valores |
|---|---|
| **StatusPedido** | EM_ANALISE, APROVADO, REPROVADO, CONTRATADO, CANCELADO |
| **ResultadoParecer** | POSITIVO, NEGATIVO |
| **Perfil** | CLIENTE, EMPRESA, BANCO |
| **TipoPropriedade** | CLIENTE, EMPRESA, BANCO |

### Ciclo de vida do pedido

Ao ser introduzido, o pedido nasce como EM_ANALISE. Após a análise financeira, passa para APROVADO (parecer positivo) ou REPROVADO (parecer negativo). Pedidos APROVADOS tornam-se CONTRATADO quando o contrato é executado. O cliente pode levar o pedido a CANCELADO a qualquer momento antes da contratação.

# 5. Diagrama de Pacotes (Visão Lógica)

O diagrama de pacotes organiza o sistema em camadas, seguindo a arquitetura MVC exigida. No back-end Spring Boot, o Controller recebe as requisições, o Service concentra as regras de negócio e o Model (entidades + repositórios) representa os dados persistidos no Supabase. No front-end React + Vite fica a View, que consome a API REST do back-end.

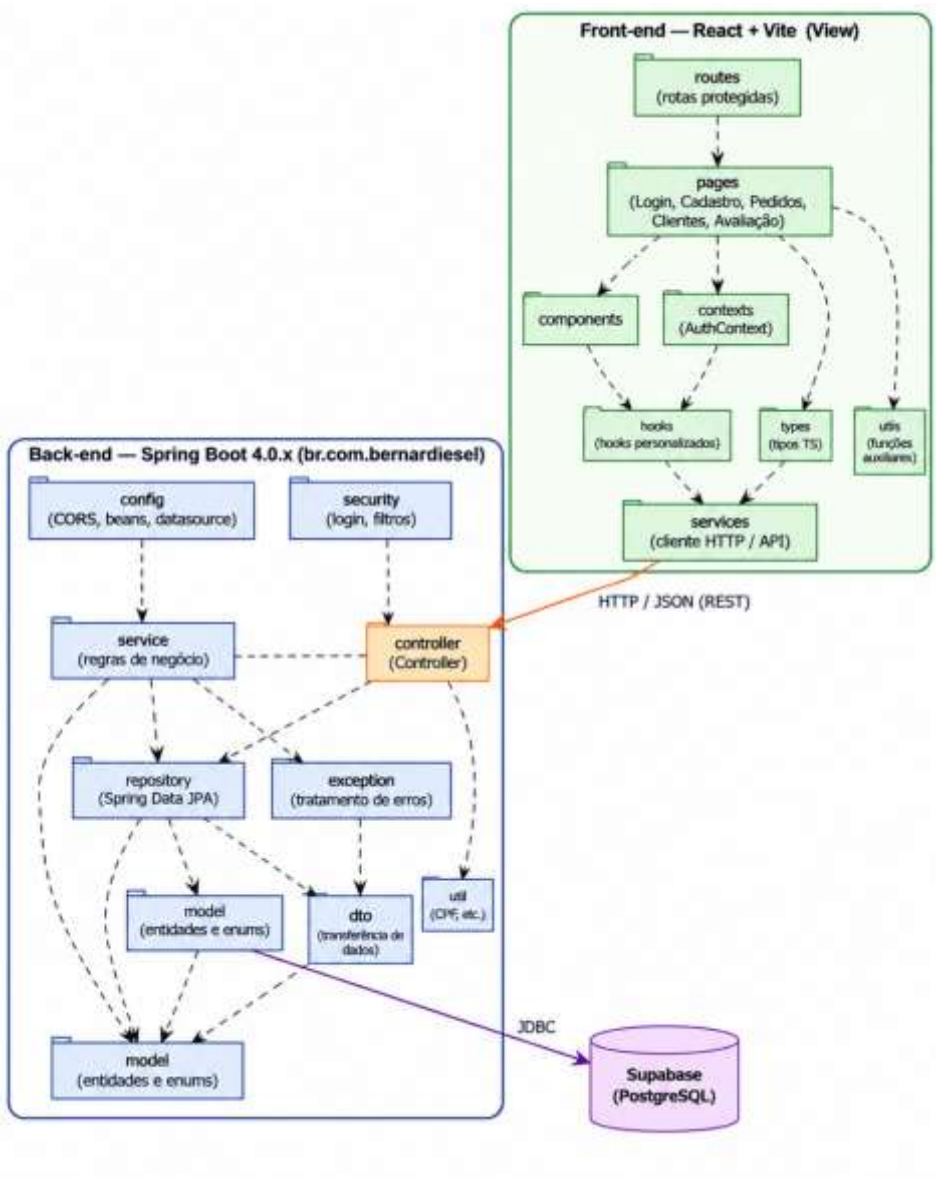



*Figura 3 – Diagrama de Pacotes (Visão Lógica) do Sistema de Aluguel de Carros*

## 5.1 Descrição dos pacotes

*Tabela 6 – Pacotes do back-end (br.com.bernardiesel)*

| Pacote | MVC | Responsabilidade |
|---|---|---|
| **controller** | Controller | Endpoints REST (clientes, pedidos, automóveis, contratos, autenticação); recebe e responde requisições. |
| **service** | Model | Regras de negócio: ciclo de vida do pedido, limite de 3 rendimentos, análise financeira, execução de contrato. |

| Pacote | MVC | Responsabilidade |
|---|---|---|
| **repository** | Model | Acesso a dados com Spring Data JPA sobre o PostgreSQL do Supabase. |
| **model** | Model | Entidades de domínio e enumerações definidas no diagrama de classes. |
| **dto** | — | Objetos de transferência de dados de entrada e saída da API. |
| **security** | — | Login (e-mail e senha), proteção de rotas e controle de acesso por perfil. |
| **config** | — | Configurações: CORS, beans, conexão com o banco e DataSeeders. |
| **exception** | — | Tratamento centralizado de erros e mensagens da API. |
| **util** | — | Métodos utilitários para normalização e validação de CPF. |

*Tabela 7 – Pacotes do front-end (React + Vite)*

| Pacote | MVC | Responsabilidade |
|---|---|---|
| **routes** | View | Definição das rotas e proteção de páginas que exigem login. |
| **pages** | View | Telas: login, cadastro, dados do cliente, pedidos, avaliação de pedidos. |
| **components** | View | Componentes reutilizáveis (formulários, tabelas, botões). |
| **contexts** | View | Estado global de autenticação (usuário logado e perfil). |
| **services** | View | Cliente HTTP que consome a API REST do back-end. |
| **hooks** | — | Hooks personalizados para reutilização de lógicas e gerenciamento de estados e contextos da aplicação. |
| **types** | — | Tipos e interfaces TypeScript utilizados pela aplicação. |
| **utils** | — | Funções utilitárias compartilhadas pelo front-end. |